B.Tech. Sem-V

Fundamentals of Operating System Design (CEUC301)

UNIT 5: CONCEPTUAL QUESTION BANK

Memory Management and Address Translation

Unit	Unit 5
Question type	Conceptual, analytical and scenario-based

Syllabus coverage: Relocation, protection, sharing and logical/physical organization; fixed and dynamic partitioning; internal/external fragmentation and compaction; first-fit, best-fit, worst-fit and next-fit; buddy allocation; paging, segmentation, address translation, page tables, multilevel/hashed/inverted page tables, and paging combined with segmentation.

A. Memory-Management Goals and Organization

1. Explain the roles of relocation, protection and sharing in a modern memory-management system.
2. Why must a process be able to execute without knowing its final physical memory location in advance?
3. Differentiate logical addresses from physical addresses. Why should applications generally use logical or virtual addresses?
4. Explain the difference between logical organization and physical organization of memory.
5. How does memory protection prevent one process from accessing another process's private memory without authorization?
6. How can memory sharing improve efficiency, and what protection mechanisms are needed when memory regions are shared?
7. Explain how hardware support such as a Memory Management Unit (MMU) contributes to relocation and protection.
8. Why are base-and-limit registers useful in some memory-management designs, and what are their conceptual limitations?
9. A system must support process isolation and shared libraries at the same time. Explain how memory-management design can provide both.

B. Fixed and Dynamic Partitioning; Fragmentation and Compaction

10. Explain fixed partitioning and dynamic partitioning, and compare how each allocates memory to processes.
11. What is internal fragmentation? Explain why fixed-size partitions can waste memory even when a process is allocated successfully.
12. What is external fragmentation? Explain how free memory can be sufficient in total but unavailable as one contiguous block.
13. Compare internal and external fragmentation in terms of their causes, symptoms and possible remedies.
14. Discuss the advantages and disadvantages of fixed partitioning with respect to simplicity, memory utilization and multiprogramming.
15. Discuss the advantages and disadvantages of dynamic partitioning with respect to flexibility, allocation overhead and fragmentation.
16. What is compaction, and how can it reduce external fragmentation?
17. Why can compaction be expensive or impractical in a system with relocation constraints, real-time requirements or large memory regions?

18. How does relocation support make it easier to move processes or memory regions during compaction?
19. Why does compaction not eliminate internal fragmentation caused by allocating a block larger than the requested amount?
20. A workload contains many small and large processes with changing lifetimes. Explain how partitioning strategy affects memory utilization and allocation success.

C. Dynamic Storage-Allocation Algorithms

21. Explain the operating principle of first-fit allocation and identify its main advantages and limitations.
22. How does best-fit select a free block, and why can it create many small unusable holes?
23. Explain worst-fit allocation and the reasoning behind choosing the largest available hole.
24. How does next-fit differ from first-fit in the way it searches for a suitable free block?
25. Compare first-fit, best-fit, worst-fit and next-fit in terms of search cost, fragmentation and likely workload sensitivity.
26. Why is it difficult to declare one dynamic storage-allocation algorithm universally best?
27. How do the free-list organization, search starting point and allocation/deallocation pattern affect allocator performance?
28. What metrics would you collect to compare dynamic storage-allocation algorithms fairly?
29. A system frequently allocates and frees blocks of varying sizes. Explain why its allocation policy should be evaluated over time rather than using a single allocation request.

D. Kernel Memory Allocation: Buddy Allocator

30. Explain the basic idea of the buddy memory-allocation system and why it divides memory into power-of-two-sized blocks.
31. How does the buddy allocator split a larger free block to satisfy a smaller allocation request?
32. Under what conditions can two free buddy blocks be merged, and why is merging useful?
33. Explain the advantages of the buddy allocator for kernel memory management, including allocation and coalescing efficiency.
34. What internal-fragmentation problem can arise when a request is rounded up to the next supported block size?
35. Compare the buddy allocator with general dynamic partitioning in terms of block sizes, fragmentation and management complexity.
36. Why can the buddy system be effective for some kernel allocation needs but less suitable for arbitrary-sized requests?
37. How does the buddy allocator's ability to split and merge blocks help manage external fragmentation?

E. Paging, Page Tables and Address Translation

38. Explain paging and how it allows a process's physical memory to be non-contiguous.
39. Define page, frame, page table, page number and page offset. Explain the role of each in address translation.
40. How is a logical address divided into a page number and offset, and how are these fields used to form a physical address?
41. Why must the page size and frame size be equal in a conventional paging system?
42. Explain how a page table maps a process's virtual pages to physical frames.
43. What information, beyond a frame number, might a page-table entry contain, and how can that information support protection or memory management?
44. How does paging eliminate external fragmentation while still potentially causing internal fragmentation?
45. What is the role of a Translation Lookaside Buffer (TLB), and why can TLB misses affect effective memory-access time?

46. Why can a page-table lookup require additional memory accesses, and how does hardware-assisted translation reduce this cost?
47. Explain how address translation and page-level protection help isolate processes while allowing selected pages to be shared.
48. What are the key trade-offs involved in choosing a page size, including page-table size, fragmentation and translation overhead?
49. Why is an example-based address-translation exercise useful for verifying understanding of page numbers, offsets and frame mapping?

F. Segmentation and Paging Combined with Segmentation

50. Explain segmentation and how segments reflect logical program units such as code, data and stack.
51. How does a logical address in segmentation differ from a logical address in paging?
52. What information is stored in a segment table, and how are bounds and protection checked during address translation?
53. Compare paging and segmentation in terms of programmer visibility, allocation, protection, sharing and fragmentation.
54. Why can segmentation support logical protection and sharing naturally, while still suffering from external fragmentation?
55. Explain the conceptual design of a system that combines segmentation with paging.
56. How can a segmented-paging scheme use a segment number to locate a page table and then use a page number and offset to locate data?
57. What additional translation steps and metadata can arise when paging is combined with segmentation?
58. Discuss the benefits and costs of combining segmentation's logical view with paging's fixed-size allocation.

G. Multilevel, Hashed and Inverted Page Tables

59. Why can a single-level page table consume excessive memory in a process with a large virtual address space?
60. Explain how a multilevel page table organizes translation metadata hierarchically.
61. How can multilevel paging avoid allocating page-table structures for unused portions of a sparse virtual address space?
62. What trade-offs does multilevel paging introduce in terms of memory use, lookup depth and implementation complexity?
63. Explain the purpose of a hashed page table and how hashing can help translate virtual page numbers in large address spaces.
64. What problems can arise from hash collisions in a hashed page table, and how can the design handle them?
65. Explain how an inverted page table differs from a conventional per-process page table.
66. Why can an inverted page table reduce the amount of memory used for translation metadata?
67. What lookup and address-space identification challenges can arise with an inverted page table?
68. Compare multilevel, hashed and inverted page tables in terms of organization, memory overhead and translation complexity.
69. Why do large address spaces and sparse memory usage influence the choice of page-table organization?

H. Integrative and Higher-Order Conceptual Questions

70. A system suffers from external fragmentation despite having enough total free memory. Explain which allocation strategies or memory-management techniques could help and what costs they introduce.
71. A process needs a large contiguous block, but free memory is scattered across many holes. Compare compaction, paging and other possible responses.

72. A kernel frequently allocates small objects of varying sizes. Explain how the buddy allocator's strengths and weaknesses should influence allocator design.
73. A process uses a very large sparse virtual address space. Compare the suitability of single-level and multilevel page tables, and discuss hashed or inverted alternatives.
74. A system requires fine-grained protection and sharing for code, data and stack regions. Compare segmentation, paging and a combined design.
75. Design a conceptual evaluation plan for comparing first-fit, best-fit, worst-fit and next-fit using representative allocation workloads and meaningful metrics.
76. Explain how memory-management choices influence performance, scalability, isolation, sharing and the overhead of address translation.
77. Critically evaluate the statement: 'Paging is superior to segmentation in every situation.' Consider logical organization, fragmentation, protection and implementation costs.